

03 — ANSWER KEY

SECTION A — MCQ answers

Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	B	11	B	21	A	31	C
2	C	12	B	22	B	32	B
3	B	13	B	23	B	33	B
4	C	14	C	24	B	34	B
5	C	15	C	25	C	35	B
6	B	16	B	26	B	36	A
7	B	17	C	27	A	37	B
8	A	18	B	28	C	38	C
9	B	19	B	29	B	39	B
10	C	20	C	30	B	40	C

Justifications for the tricky ones: - **2 – C (Generic):** COTS = sold on the open market to many users; "Buy", not "Build". - **3 – B:** SQA prevents defects and does **not** execute the program; executing/detecting is SQC. - **5 vs 6:** *Error* = human action; *Failure* = inability to perform the function. Don't swap them. - **12 / 13:** Reliability = **factor** (behavioural characteristic); Modularity = **criterion** (attribute of a factor). - **14 – C:** Security is **not** an ISO-9126 top-level characteristic (the six are Functionality, Reliability, Usability, Efficiency, Maintainability, Portability). - **18 / 19:** Level 4 = **Managed**, Level 5 = **Optimizing** — a classic reversal trap. - **25 / 26:** Auto-save and NVP are **containment**, not prevention. - **30 – B:** Test plan = high-level; test case = low-level. - **32 – B:** Regression **reuses** existing tests; it does not design new ones. - **36 – A:** Principle 1 = presence, not absence. - **40 – C:** Usability testing cannot be fully automated.

SECTION B — True/False answers

Q	Ans	Why (if tricky)
1	T	SQC always executes the program.
2	F	In SQA all team members are responsible; "only testing team" is SQC.
3	F	Dynamic analysis executes the program; static does not.
4	F	That defines an error , not a fault.
5	F	Security is not an ISO-9126 top-level characteristic.
6	T	Manufacturing view = conformance to requirements.
7	F	Level 5 = Optimizing ; Level 4 = Managed.
8	T	TMM has 5 levels.
9	F	Prevention is never 100% effective.
10	T	NVP is a fault-tolerance / containment technique.
11	F	Test case is low-level .
12	F	Regression reuses tests; designs none.
13	T	Principle 1.
14	F	Exhaustive testing is nearly impossible.
15	T	Acceptance testing = end-user.
16	T	WBT is done mainly by developers.
17	F	Automation cannot replace manual testing.
18	F	Reliability is a factor , not a criterion.
19	T	Kitchenham & Pfleeger.
20	F	That's an accident ; a hazard is a pre-condition to an accident.

SECTION C — Model answers

(Write them in this shape: definition → list/structure → example. Mark split shown in brackets.)

C1 — SQA vs SQC (8)

Definition [2]: SQA (Software Quality Assurance) is a planned, systematic, process-oriented set of activities that assures the standards, processes and procedures are appropriate and correctly implemented — an umbrella activity across the whole SDLC. **SQC (Software Quality Control)** is a product-oriented set of activities that checks the project follows those standards and produces the required deliverables, by executing/evaluating the work product.

Differences [5] (any five):

SQA	SQC
Aim: prevent defects	Aim: identify & resolve defects
Does not execute the program	Always executes the program
All team members responsible	Only the testing team responsible
Maps to Verification (doing things right)	Maps to Validation (results as expected)
Failure prevention system; full SDLC	Failure detection system; testing life cycle

Verification/Validation [1]: SQA = **Verification**; SQC = **Validation**.

Example: Defining the coding standard and review checklist for a login module = **SQA**; actually running the login test cases and logging the bugs = **SQC**.

C2 — Five views of quality (Kitchenham & Pfleeger) (8)

Definition [1]: Kitchenham and Pfleeger describe quality from five different perspectives because different stakeholders judge quality differently.

The five views [5] + examples [2] (≈1.5 each): 1. **Mystical view** — quality is recognized through experience but cannot be defined; a good product "stands out". *Example:* "I know good software when I see it." 2. **User view** — quality = the extent to which the product meets user needs/expectations; good if it satisfies many users (usability, reliability, efficiency). *Example:* an app rated highly by most users. 3. **Manufacturing view** — conformance to process standards/requirements; "right the first time"; improve the product by improving the process. *Example:* CMM/ISO 9001-based development. 4. **Product view** — good **internal** properties produce good **external** properties. *Example:* clean modular code → fewer field failures. 5. **Value-based view** — customers' willingness to pay; merges **excellence** (quality) and **worth** (value); trades cost off against quality. *Example:* a "good enough for the price" product.

(Mnemonic: My Uncle Makes Pretty Vases.)

C3 — CMM five levels + how TMM/TPI differ (8)

Definition [1]: The CMM measures how mature an organization's software **development** process is — i.e. to what extent it can produce low-cost, high-quality software.

Five levels [5]: 1. **Initial** — processes disorganized/chaotic; success depends on individuals; not repeatable. 2. **Repeatable** — basic project management established; successes can be repeated. 3. **Defined** — the organization has its own standard process (documentation, standardization, integration). 4. **Managed** — processes monitored and controlled via **data collection and analysis**. 5. **Optimizing** — processes continuously improved via feedback and innovation.

(Mnemonic: Initial, Repeatable, Defined, Managed, Optimizing.)

How TMM/TPI differ [2]: - **TMM** (pioneered by **Ilene Burnstein**) also has 5 levels but **evaluates/improves the testing process**, not development. - **TPI** is a model specifically for **improving** the testing process (4 steps: determine area → evaluate current state → identify next state → implement changes).

Example: A company that only tests after coding, ad hoc, with no tracking, sits at **CMM Level 1 / TMM Level 1**.

C4 — Three QA categories (8)

Definition [1]: QA focuses on the correctness aspect of quality; its activities fall into three generic categories that deal with defects at different points.

The three categories [4] + examples [1]: 1. **Defect Prevention** — stop faults from being **injected** in the first place. *Example:* providing default values / valid options so wrong input can't be entered. 2. **Defect Reduction** — **detect and remove** faults that have already been injected. *Example:* inspection and testing. 3. **Defect Containment** — **contain failures** to local areas or **limit the damage** to assure reliability/safety. *Example:* auto-save / exception handling.

Two techniques each [2]: - **Defect prevention:** ① **error source removal** (education & training — people-based) ② **error blocking** (e.g. don't allow a 0 divisor; data validation). - **Defect containment:** ① **software fault-tolerance** (Recovery = rollback & redo; **NVP** = N-Version Programming) ② **safety assurance & failure containment** (hazard analysis, damage control).

(Note: prevention is never 100% effective — that's why reduction and containment exist.)

C5 — Test plan/case/suite + WBT vs BBT (8)

(a) Definitions [3]: - **Test plan** — a **high-level** document describing the objectives, scope, approach, resources, schedule and focus of testing. - **Test case** — a **low-level** document describing one input/action/event and its expected result, to check if a feature works. - **Test suite** — the **macro-level** collection of individual test cases run in sequence until stopping criteria are met (reusing earlier cases = regression testing).

(b) White-box vs Black-box [4] + examples [1]:

Dimension	White-box (structural)	Black-box (functional)
Basis	implementation / source code	specification / external interface
Focus	control flow & data flow	input–output behaviour
Object size	small units	large systems
Timeline	early (unit/component)	late (system/acceptance)
Tester	developers	professional / third-party (IV&V)

Example: Checking every branch of a function's `if/else` = **white-box**; entering a username/password on the login screen and checking the result without seeing the code = **black-box**. (*Middle levels = gray-box.*)